

Tcl tool command language

- TCL is a scripting language
- To run tcl we use cygwin
- cygwin -it is software,simulator,emulator .To run the scripting languages and linux commands also
- #!what is this in cygwin?
- In Cygwin, **#! (called shebang or hashbang)** is a special sequence of characters that appears at the beginning of a script file and is used to specify the interpreter that should be used to execute the script.
- For example, if you have a Tcl script named "myscript.tcl" and you want to execute it using the Tcl interpreter, you can add the following shebang line at the top of the file:
- **#!/usr/bin/tclsh**
- This tells the operating system that the script should be executed using the tclsh interpreter located at /usr/bin/tclsh.
- When you run the script from the command line, the operating system uses the shebang line to determine the appropriate interpreter to use For example:
- **./myscript.tcl or we use tclsh filename.tcl** To run the code into the file
- In this case, the operating system looks at the shebang line in "myscript.tcl" and
- uses /usr/bin/tclsh to execute the script.
- session: false expression
- The formal syntax of TCL (Tool Command Language) is based on commands, which are a sequence of words that specify an operation to be performed. Each command in TCL has the following syntax:
- **commandName arg1 arg2**
- **set a 20**
- **formal syntax of the tcl language : command -options/flags arg1 arg2 arg3.....argn**
- **ex:- set a 10**
- here **set** is command a,10 is arguments
- In interactive mode tcl code executed line by line
- For this we type tclsh in Cygwin it shows
- **%** for tclsh interpreter
- **\$** for bash interpreter
- **>>>** for python interpreter
- **EX- set a 10**
- **10**
- **Set b 10**
- **10**
- **Expr :-** is used to run the arithmetic expressions
- TCL is a dynamic language In which means that data types are specified during runtime based on the value assigned to a variable There is no explicit declarations present in TCL,
- **Data types:-**
Int
Float,lists,arrays ,strings,dictionary

Tcl tool command language

Uses :-

`;` :- terminate the line

`{ }` :- it is used to group the words into single arguments

`[ ]` :-to evaluate and execute a command and assign value to variable

Difference b/n `[ ]` & `{ }`

EX:-set a 10

set b 20

set [expr \$a+\$b] --o/p shows -30

set {expr \$a+\$b} --o/p shows expr \$a+\$b

Operators:-

1.uninarry operators

2. binary operators

3. terinary operators

1.Uninary operators

In this we have inly one operand

Increment(+5),decrement(-1) ,logical not(!) and Bitwise(~) operators

2. Binary operators

In this we have to operands

1.Arithmetic operators :- {+, -, *, /, %, **}

2.Relational operators :- {> , < ,<=,>=,==,!=}

3.Logical operators :- && ,||

4.bitwise operators :- (bit wise and)&, | (or) , ^ (xor) , ~&(nand) , ~|(nor) , ~^ (xnor)

5.shift operators :- >> (right shift) ,<< (left shift)

In right shift number div by 2**n

Tcl tool command language

In left shift num mul by 2^{**n}

3.Ternary operators :-

(condition) ? true expression :false expression

##control structers:

1.if statement

2.if else statement

3.if elseif statement

4.nested statement

Ex if:-

```
if {condition} {  
    body of if  
}
```

Ex if else:-

```
if {condition} {  
    body of if  
} else {  
    body of else  
}
```

Ex else if:-

```
if {condition} {
```

Tcl tool command language

body of if

} elseif {condition} {

body of elseif

} else {

body of else

}

##multi branching statements :-

1. switch

syntax for switch:switch {condition} {

"case1" {body of the case1}

"case2" {body of the case2}

"case3" {body of the case3}

default {body of the default}

}

##ex programm

set a "apple"

switch \$a {

"apple" {puts "it is a apple"}

"banana" {puts "it is banana"}

default {puts "it is default case"}

}

Tcl tool command language

##LOOPS :-

1.for loop [break it exist the loop, continue it stops the loop at certain condyion]

2.while loop

3.foreach loop

1.for loop:-

```
for {intilization } {condition} {incr/decr} {  
    body of for loop  
}
```

2. while loop:

```
(intilization{set i 1}  
while{condtion} {  
    body of while loop  
    incr/decr  
}
```

#ex program

```
set i 1  
while {$i<=100} {
```

Tcl tool command language

```
if {$i/6==10} {  
    puts $i  
    incr i  
    break;  
}  
}
```

Exmample programs for break ,continue :

###program for factorial of a giuve number user prompted input

```
puts $argv  
lassign $argv a  
set fact 1  
for {set i 1} {$i<=$a} {incr i} {  
    set fact [expr $fact *$i]  
}  
puts $fact
```

program for print 1 -100 num which is div by 4 & 6 stop when it is in 70

```
for {set i 1} {$i<=100} {incr i} {  
    if {$i%4==0 && $i%6==0} {  
        if {$i/7==10} {  
            break  
        }  
        puts $i  
    }  
}
```

program for print 1 -100 num which is div by 4 & 6 skip when it is in 70

Tcl tool command language

```
for {set i 1} {$i<=100} {incr i} {  
  if {$i%4==0 && $i%6==0} {  
    if {$i/7==10} {  
      continue  
    }  
    puts $i  
  }  
}
```

###foreach :

def:-it is used to iterate the entire list

syntax:-

```
foreach variable $list {  
  puts $variable #it can print each an every variable in list  
}
```

example 1:

write a tcl program to print all the prime numbers from 2 to 100.

Loop to check and print prime numbers from 2 to 100

```
for {set j 2} {$j <= 100} {incr j} {  
  set isPrime 1  
  for {set i 2} {$i < $j} {incr i} {  
    if {$j % $i == 0} {  
      set isPrime 0  
      break  
    }  
  }  
  if {$isPrime} {  
    puts $j  
  }  
}
```

Tcl tool command language

example 2:

write a tcl program to find the factorial of the number ?

```
set fact 1
for {set i 0} {$i <= 16} {incr i} {
puts "$i! = $fact"
set fact [expr {$fact * ($i + 1)}]
}
```

for loop with break and continue :

example 3:

write a tcl program to print all the numbers from 1 to 100.
and break when in its in 70's.

```
for {set i 1} {$i <= 100} {incr i} {
if {$i / 10 == 7} {
break ; # stop the iteration or termination of the loop
}
puts $i
}
```

example 4:

write a tcl program to print all the numbers from 1 to 100.
and skip when in its in 70's.

```
for {set i 1} {$i <= 100} {incr i} {
if {$i / 10 == 7} {
continue ; # skip current iteration and continue
}
puts $i
}
```

example 5:

write a tcl program to print all the numbers from 1 to 100.
which is divisible by 4 and stop when its in 70's.

```
for {set i 1} {$i <= 100} {incr i} {
if {$i % 4 == 0} {
if {$i / 10 == 7} {
break
}
```


Tcl tool command language

```
}  
puts $i  
}  
}
```

example 6:

write a tcl program to print all the numbers from 1 to 100.
which is divisible by 4 and skip when its in 70's.

```
for {set i 1} {$i <= 100} {incr i} {  
  if {$i % 4 == 0} {  
    if {$i / 10 == 7} {  
      continue  
    }  
    puts $i  
  }  
}
```

example 7:

write a tcl program to print all the numbers from 1 to 100.
which is divisible by 4 and skip when its in 70's and 90's.

```
for {set i 1} {$i <= 100} {incr i} {  
  if {$i % 4 == 0} {  
    if {$i / 10 == 7 || $i / 10 == 9} {  
      continue  
    }  
    puts $i  
  }  
}
```

example 8:

write a tcl program to print all the numbers from 1 to 100.
which is divisible by 4 and skip when its in 60's and stop when it's in 90's.

```
for {set i 1} {$i <= 100} {incr i} {  
  if {$i % 4 == 0} {  
    if {$i / 10 == 6} {  
      continue  
    }  
    if {$i / 10 == 9} {  
      break  
    }  
  }  
}
```

Tcl tool command language

```
puts $i  
}  
}
```

while loop :

syntax of the while loop :

```
set i 0  
while {condition} {  
  body of the while loop  
  incr i  
}
```

example 9:

write a tcl program to print all the even numbers from 0 to 100.
using while loop.

```
set i 0  
  
while {$i <= 100} {  
  puts $i  
  incr i 2  
}
```

example 9:

write a tcl program to print all the even numbers from 1 to 100.
using while loop.

```
set i 1  
  
while {$i <= 100} {  
  if {$i % 2 == 0} {  
    puts $i  
  }  
  incr i  
}
```

example 10:

write a tcl program to print all the odd numbers from 1 to 100.
using while loop.

Tcl tool command language

```
set i 1
while {$i <= 100} {
  puts $i
  incr i 2
}
```

example 11:

write a tcl program to print all the odd numbers from 1 to 100.
using while loop and skip when its in 60's.

```
set i 1

while {$i <= 100} {

  if {$i / 10 == 6} {
    incr i 2
    continue
  }
  puts $i
  incr i 2
}
```

example 12:

write a tcl program to print all the odd numbers from 1 to 100.
using while loop and skip when its in 60's and 80's.

```
set i 1

while {$i <= 100} {

  if {$i / 10 == 6 || $i / 10 == 8} {
    incr i 2
    continue
  }
  puts $i
  incr i 2

}
```

example 13:

write a tcl program to print all the even numbers from 1 to 100.
using while loop and break when its in 60's.

Tcl tool command language

```
set i 1

while {$i <= 100} {

if {$i % 2 == 0 } {
puts $i
if {$i / 10 == 6} {
break
}
}
incr i
}
```

switch statement :

syntax :

```
switch $var {
"case1" {
puts "print case1"
}
"case2" {
puts "print case2"
}
"case3" {
puts "print case3"
}
default {
puts "default body"
}
}
```

example 14 :

write a tcl program to check the mango is present in the list are not.

work on to this program

```
puts "Enter the fruit name:"
```

```
gets stdin b
```

```
set fruits {apple mango banana cherry}
```

```
#puts $fruits
```

Tcl tool command language

```
foreach a $fruits {  
#puts $a  
}
```

```
if {$a == $b} {
```

```
switch $b {  
"apple" { puts "Apple is present" }  
"bannana" { puts "bannana is present"}  
"mango" { puts "mango is present" }  
default {puts "No fruits present"}  
}  
}
```

for loop :

syntax of the for loop in tcl:

```
for {initialization} {condition} {incr/decrement} {  
body of the for loop  
}
```

```
for {set i 1} {$i <= 100} {incr i 2} { puts $i }
```

```
for {set i 1} {$i <= 100} {incr i} {  
puts $i  
}
```

1. write a tcl pgm to print all the odd numbers from 1 to 100 using for loop.

```
for {set i 1} {$i <= 100} {incr i 2} {  
puts $i  
}
```

2. write a tcl pgm to print all the even numbers from 1 to 100 using for loop.

```
for {set i 2} {$i <= 100} {incr i 2} {  
puts $i  
}
```

3. write a tcl pgm to print all the numbers from 100 to 1 using for loop.

```
for {set i 100} {$i >= 1} {incr i -1} {  
puts $i  
}
```

Tcl tool command language

4. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 using for loop.

```
for {set i 4} {$i <= 100} {incr i 4} {  
  puts $i  
}
```

for(i=1;i<=10;i++) --> C lang

for x in range(11):
 print(x) --> python

if statment:

```
if {condition} {  
  body of the if stataement  
}
```

```
set a 10  
set b 10
```

```
if {$a == $b} {  
  puts "a is equal to b"  
}
```

```
set a 10  
set b 10
```

```
if {$a == $b} {  
  puts "a is equal b"  
} else {  
  puts "a is not equal to b"  
}
```

```
set a 5  
set b 10
```

```
if {$a > $b} {  
  puts "a is greater than b"  
} elseif {$a < $b} {  
  puts "a is less than b"  
} else {  
  puts "a is equal to b"  
}
```

Tcl tool command language

5. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 using for loop and if statement.

```
for {set i 1} {$i <= 100} {incr i 1} {  
  if {$i % 4 == 0} {  
    puts $i  
  }  
}
```

6. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 and divisible by 6 using for loop and if statement.

```
for {set i 1} {$i <= 100} {incr i 1} {  
  if {$i % 4 == 0 && $i % 6 == 0} {  
    puts $i  
  }  
}
```

for loop with break statement:
used to break the loop

```
for {set i 1} {$i <= 100} {incr i 1} {  
  if {$i == 70} {  
    break  
  }  
  puts $i  
}
```

for loop with continue statement:
it is used to skip the particular iteration

```
for {set i 1} {$i <= 100} {incr i 1} {  
  if {$i == 70} {  
    continue  
  }  
  puts $i  
}
```

7. write a tcl pgm to print all the numbers from 1 to 100 which when it is in 70's skip it using for loop and if statement.

Tcl tool command language

```
for {set i 1} {$i <= 100} {incr i 1} {  
  if {$i / 10 == 7} {  
    continue  
  }  
  puts $i  
}
```

7. write a tcl pgm to print all the numbers from 1 to 100 which when it is in 70's and 80's skip it using for loop and if statement.

```
for {set i 1} {$i <= 100} {incr i 1} {  
  if {$i / 10 == 7 || $i / 10 == 8} {  
    continue  
  }  
  puts $i  
}
```

while loop :

```
set i 1  
while {condition} {  
  
  body of the while loop  
  incr i  
}
```

8. write a tcl pgm to print all the numbers from 1 to 100 using while loop.

```
set i 1  
  
while {$i <= 100} {  
  puts $i  
  incr i  
}
```

9. write a tcl pgm to print all the even numbers from 1 to 100 using while loop.

```
set i 2  
  
while {$i <= 100} {  
  puts $i
```


Tcl tool command language

```
incr i 2  
}
```

Assignment:

work with break statement in while loop.

```
set i 1  
while {$i <= 10} {  
    if {$i == 5} {  
        break  
    }  
    puts $i  
    incr i  
}
```

work with continue statement in while loop.

```
set i 1  
while {$i <= 10} {  
    if {$i == 5} {  
        incr i  
        continue  
    }  
    puts $i  
    incr i  
}
```

10. write a tcl pgm to print all the odd numbers from 1 to 100 using while loop.

```
set i 1  
while {$i <= 100} {  
    if {$i % 2 == 1} {  
        puts $i  
    }  
    incr i  
}
```

11. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 using while loop.

```
set i 1  
while {$i <= 100} {  
    if {$i % 4 == 0} {
```

Tcl tool command language

```
    puts $i
  }
  incr i
}
```

12. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 and 6 using while loop.

```
set i 1
while {$i <= 100} {
  if {$i % 4 == 0 && $i % 6 == 0} {
    puts $i
  }
  incr i
}
```

13. write a tcl pgm to print all the numbers from 1 to 100 which when it is in 70's and 80's skip it using while loop and if statement.

```
set i 1
while {$i <= 100} {
  if {$i >= 70 && $i <= 89} {
    incr i
    continue
  }
  puts $i
  incr i
}
```

14. write a tcl pgm to find the factriol of the number.

```
puts "Enter a number to find the factorial:"
set a [gets stdin]
```

```
set b 1
```

```
for {set i 1} {$i <= $a} {incr i} {
  set b [expr $i*$b]
}
puts "the factorial of the number is:$b"
```

15. write a tcl pgm to print all the prime number from 2 to 100.

```
for {set i 2} {$i <= 100} {incr i} {
  set is_prime 1
  for {set j 2} {$j < $i} {incr j} {
```

Tcl tool command language

```
if {$i % $j == 0} {  
    set is_prime 0  
    break  
}  
}  
if {$is_prime == 1} {  
    puts $i  
}  
}
```

switch statement:

```
switch $var {  
    "case1" {  
        body of the case1  
    }
```

```
    "case2" {  
        body of the case2  
    }
```

```
    "case3" {  
        body of the case3  
    }  
    default {  
        body of the default  
    }  
}
```

```
set fruit "mango"
```

```
switch $fruit {  
    "apple" {  
        puts "apple is present"  
    }  
    "bananna" {  
        puts "bananna is present"  
    }  
    "orange" {  
        puts "orange is present"  
    }  
    default {  
        puts "apple,bananna and orange is not present"    }  
}
```

Tcl tool command language

```
}  
}
```

while loop:

```
set i 1
```

```
while {condition} {  
  body of the while  
  incr i  
}
```

switch statments:

```
set a "apple"
```

```
switch $a {  
  "apple" {  
    body of the case-1  
  }  
  "orange" {  
    body of the case-2  
  }  
  default {  
    body of the default  
  }  
}
```

work with break statement in while loop.

1. write a tcl program to print the numbers from 1 to 100. stop the iteration with its at 60 using while loop.

```
set i 1
```

```
while {$i <= 100} {  
  if {$i == 60} {  
    break  
  }  
  puts $i  
  incr i  
}
```

work with continue statement in while loop.

2. write a tcl program to print the numbers from 1 to 100. skip if its in 60's using while loop.

Tcl tool command language

```
set i 1
```

```
while {$i <= 100} {  
  if {$i / 10 == 6} {  
    incr i  
    continue  
  }  
  puts $i  
  incr i  
}
```

work with continue statement in while loop.

3. write a tcl program to print the numbers from 1 to 100. skip if its in 60's and 80's using while loop.

```
set i 1
```

```
while {$i <= 100} {  
  if {$i / 10 == 6 || $i / 10 == 8} {  
    incr i  
    continue  
  }  
  puts $i  
  incr i  
}
```

10. write a tcl pgm to print all the odd numbers from 1 to 100 using while loop.

```
set i 1  
while {$i <= 100} {  
  puts $i  
  
  incr i 2  
  
}
```

```
set i 1  
while {$i <= 100} {  
  if {$i % 2 == 1} {  
    puts $i  
  }  
  incr i  
}
```

Tcl tool command language

11. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 using while loop.

```
set i 1
while {$i <= 100} {
if {$i % 4 == 0} {
puts $i
}
incr i
}
```

12. write a tcl pgm to print all the numbers from 1 to 100 which is divisible by 4 and 6 using while loop.

```
set i 1
while {$i <= 100} {
if {$i % 4 == 0 && $i % 6 == 0} {
puts $i
}
incr i
}
```

13. write a tcl pgm to print all the numbers from 1 to 100 which when it is in 70's and 80's skip it using while loop and if statement.

```
set i 1

while {$i <= 100} {
if {$i / 10 == 7 || $i / 10 == 8} {
incr i
continue
}
puts $i
incr i
}
```

14. write a tcl pgm to find the factorial of the given number.

note: user providing the number.

```
puts "Enter the number:"
set a [gets stdin]
```

Tcl tool command language

```
set b 1
for {set i 1} {$i <= $a} {incr i 1} {
    set b [expr $i*$b]
}
puts "the factorial of the given number is:$b"
```

15. write a tcl pgm to print all the prime number from 2 to 100.

```
for {set i 2} {$i <= 100} {incr i} {

    set prime 1

    for {set j 2} {$j < $i} {incr j} {
        if {$i % $j == 0} {
            set prime 0
            break
        }
    }

    if {$prime == 1} {
        puts $i
    }
}
```

- **STRINGS;-**

DEF;-It is collection of characters or sequence of characters

STRING manipulation Techniques;-

- 1.length
- 2.reverse
- 3.range
- 4.map
- 5.toupper
- 6.tolower
- 7.first
- 8.last
- 9.trimleft
- 10.trimright

11 trim

n TCL string is enclosed by double quotes ("")

string contains in TCL characters, numbers, special characters.

how to define the string in the TCL?

```
set a "this is tcl class"
```

```
puts $a
```

In TCL, We have some built in string operators:

1. string length:

it is used to find the length of the string including spaces.

```
set a "this is tcl class"
```

```
set b [string length $a]
```

```
puts $b
```

2. string reverse

it is used to reverse the string

```
set a "this is tcl class"
```

```
set b [string reverse $a]
```

```
puts $b
```

3. string compare:

Compares two strings lexicographically and returns -1 if the first string is less than the second, 0 if they are equal, and 1 if the first string is greater than the second.

```
# String compare if string1 and string2 is same, prints output is 0
```

```
set a "this is tcl class"
```

```
set b "this is tcl class"
```

```
set c [string compare $a $b]
```

```
puts "its prints output is:$c"
```

```
# String compare if string1 less than and string2 , prints output is -1
```

```
set a "this is tcl cla"
```

```
set b "this is tcl class"
```


Tcl tool command language

```
set c [string compare $a $b]
puts "its prints output is:$c"
```

```
# String compare if string1 greater than and string2 , prints output is 1
set a "this is tcl classs"
set b "this is tcl class"
```

```
set c [string compare $a $b]
puts "its prints output is:$c"
```

1. string length:

```
Tcl
set str "Hello, world!"
set length [string length $str]
puts "Length of the string: $length" # Output: 13
Use code with caution.
```

2. string reverse:

```
Tcl
set str "abcde"
set reversed_str [string reverse $str]
puts "Reversed string: $reversed_str" # Output: edcba
```

3. string compare:

```
Tcl
set str1 "hello"
set str2 "world"
set comparison_result [string compare $str1 $str2]
if {$comparison_result == 0} {
    puts "Strings are equal"
} else {
    puts "Strings are not equal"
} # Output: Strings are not equal
```

4. string index:

```
Tcl
set str "abcde"
set char [string index $str 2]
puts "Character at index 2: $char" # Output: c
```

5. string range:

```
Tcl
set str "Hello, world!"
set substring [string range $str 7 11]
puts "Substring: $substring" # Output: world
```

Tcl tool command language

6. string append:

```
Tcl
set str "Hello"
string append str " world!"
puts "Appended string: $str" # Output: Hello world!
```

7. string tolower:

```
Tcl
set str "Hello, World!"
set lowercase_str [string tolower $str]
puts "Lowercase string: $lowercase_str" # Output: hello, world!
```

8. string toupper:

```
Tcl
set str "Hello, World!"
set uppercase_str [string toupper $str]
puts "Uppercase string: $uppercase_str" # Output: HELLO, WORLD!
```

9. string trim:

```
Tcl
set str " Hello, world! "
set trimmed_str [string trim $str]
puts "Trimmed string: $trimmed_str" # Output: Hello, world!
```

10. string trimleft:

```
Tcl
set str " Hello, world!"
set trimmed_str [string trimleft $str]
puts "Left-trimmed string: $trimmed_str" # Output: Hello, world!
```

11. string trimright:

```
Tcl
set str "Hello, world! "
set trimmed_str [string trimright $str]
puts "Right-trimmed string: $trimmed_str" # Output: Hello, world!
```

12. string first:

```
Tcl
set str "Hello, world!"
set position [string first "world" $str]
puts "Position of 'world': $position" # Output: 7
```

13. string last:

Tcl tool command language

Tcl

```
set str "Hello, world! Hello again!"
set position [string last "Hello" $str]
puts "Position of the last 'Hello': $position" # Output: 16
```

14. string wordstart:

Tcl

```
set str "Hello, world!"
set start_position [string wordstart $str 2]
puts "Start position of the second word: $start_position" # Output: 7
```

15. string wordend:

Tcl

```
set str "Hello, world!"
set end_position [string wordend $str 2]
puts "End position of the second word: $end_position" # Output: 11
```

16. string equal:

Tcl

```
set str1 "hello"
set str2 "world"
if {[string equal $str1 $str2]} {
    puts "Strings are equal"
} else {
    puts "Strings are not equal"
} # Output: Strings are not equal
```

17. string is:

Tcl

```
set str "hello"
if {[string is alpha $str]} {
    puts "String contains only alphabetic characters"
} # Output: String contains only alphabetic characters
```

18. string match:

Tcl

```
set str "hello world"
if {[string match "hello*" $str]} {
    puts "String starts with 'hello'"
} # Output: String starts with 'hello'
```

19. string map:

Tcl

```
set str "abcde"
```

Tcl tool command language

```
set mapped_str [string map {a b} {1 2} $str]
puts "Mapped string: $mapped_str" # Output: 12cde
```

20. string replace:

```
Tcl
set str "Hello, world!"
set replaced_str [string replace $str 7 11 "there"]
puts "Replaced string: $replaced_str" # Output: Hello, there!
```

21. string cat:

```
Tcl
set str1 "Hello"
set str2 " world!"
set concatenated_str [string cat $str1 $str2]
puts "Concatenated string: $concatenated_str" # Output: Hello world!
```

22. string join:

```
Tcl
set list {"Hello" "world!"}
set joined_str [string join " " $list]
puts "Joined string: $joined_str" # Output: Hello world!
```

23. string format:

```
Tcl
set name "Alice"
set age 30
set formatted_str [string format "My name is %s and I am %d years old." $name $age]
puts "Formatted string: $formatted_str" # Output: My name is Alice and I am 30 years old.
```

Discussion problems:-

Discussion 1:

```
puts "Enter the fruit name:"
gets stdin b
```

```
set fruits {apple mango banana cherry}
```

```
set isPresent 0
foreach a $fruits {
    if {$a == $b} {
```

Tcl tool command language

```
        set isPresent 1
        break
    }
}

if {$isPresent} {
    switch $b {
        "apple" { puts "Apple is present" }
        "banana" { puts "Banana is present" }
        "mango" { puts "Mango is present" }
        default { puts "Fruit is present, but not recognized" }
    }
} else {
    puts "Fruit is not present"
}
```

Discussion 2 :

```
set sentence "Hello, how are you?"
```

```
# Find the end position of the word "how"
```

```
set endPos [string wordend $sentence [string first "how" $sentence]]
puts "End position of 'how': $endPos" ;# Output: End position of 'how': 5
```

```
# Find the end position of the first word
```

```
set endPos [string wordend $sentence 0]
puts "End position of first word: $endPos" ;# Output: End position of first word: 4
```

```
# Find the end position of the second word starting from index 7
```

```
set endPos [string wordend $sentence 9]
puts "End position of second word: $endPos" ;# Output: End position of second word: 9
```

```
#set sentence "Hello, how are you?"
```

```
# Find the end position of the word "how"
```

```
set endPos [string wordend "Now is the time" 7]
puts "End position of 'how': $endPos" ;# Output: End position of 'how': 5
```

Discussion 3 :

```
set str "a1"
```

```
if {[string match {[a-z][0-9]*} $str]} {
    puts "$str contains 'a1'"
} else {
```

Tcl tool command language

```
puts "$str does not contain 'a1'"  
}
```

20.string map

In Tcl, string map is a command that is used to replace occurrences of one or more substrings in a string with a specified replacement string. The basic syntax of string map is:

```
set str "The quick brown fox jumped over the lazy dog."  
set newstr [string map {quick fast brown red fox dog} $str]  
puts $newstr
```

```
set str "The quick brown fox jumped over the lazy dog."  
puts $str  
set newstr [string map {" " ""} $str]  
puts $newstr
```

21. string replace

In Tcl, string replace is a command that is used to replace a portion of a string with a specified replacement string.

```
set str "The quick brown fox jumped over the lazy dog."  
puts $str  
set newstr [string replace $str 4 8 "slow"]  
puts $newstr
```

22. string cat

In Tcl, string cat is a command that concatenates one or more strings together into a single string.

```
set str1 "The quick brown fox "  
set str2 "jumped over the lazy dog."  
set sentence [string cat $str1 $str2]  
puts $sentence
```

23. string repeat

In Tcl, string repeat is a command that is used to repeat a string a specified number of times.

```
set str "hello"  
set repeated_str [string repeat $str 3]  
puts $repeated_str
```

Tcl tool command language

24. string join

In Tcl, string join is a command that is used to concatenate a list of strings into a single string, with a specified separator between each string.

```
set mylist {apple banana orange}
set separator ","
set joined_str [lindex $mylist 0]
for {set i 1} {$i < [llength $mylist]} {incr i} {
    append joined_str $separator [lindex $mylist $i]
}
puts $joined_str
```

25. string format

In Tcl, string format is a command that is used to create a formatted string based on a format string and a list of values.

```
set name "John"
set age 30
set height 1.8
set formatted_str [format "Name: %s, Age: %d, Height: %.2f meters" $name $age
$height]
puts $formatted_str
```

example 1:

write a tcl program to find the length of the string including spaces. ?

```
set a "this is tcl class"
set b [string length $a]
puts "The length of the string is:$b"
```

example 2:

write a tcl program to find the length of the string excluding white spaces.?

```
set text "welcome to bangalore and welcome to mumbai"
```

```
set c [string length $text]
puts "length of the string including white spaces:$c"
```

```
set a [string map {" " ""} $text]
set b [string length $a]
puts "length of the string excluding white spaces:$b"
```

Tcl tool command language

example 3:

write a tcl program to find the number of the words present in string ?

```
set text "welcome to bangalore and welcome to mumbai"
```

```
foreach a $text {  
  puts $a  
}
```

example:1

find the number of character of the text(including spaces)

```
set a "welcome to bangalore and welcome to mumbai"
```

```
set b [string length $a]
```

```
puts "the number of characters including space is:$b"
```

example:2

find the number of character of the text(Excluding spaces)

```
set text "This is a sample text."
```

```
set text_nospace [string map {" " ""} $text]
```

```
set length_nospace [string length $text_nospace]
```

```
puts "The length of the text excluding spaces is: $length_nospace"
```

example:3

find the number of words in a text.

```
set a "welcome to bangalore and welcome to mumbai welcome  
to delhi"
```

```
set i 0  
foreach b $a {  
  #puts $b  
  incr i  
}
```

```
puts "the number of the words in a text is:$i"
```

In TCL, a list is a collection of values, typically strings or numbers, that are stored as a single value. Lists are represented as a series of elements separated by white space or brackets.

```
set a {1 2 3 4 5 6 6}  
puts $a
```


LISTS ;-

list in the TCL:

list is collection of items or elements which are in ordered.

list is a one of the basic data-type in the TCL.

list can contain element of any type.

list can allow to create a list in the list.

how to create list.

```
set a "1 2 3 4 5 6"
```

```
set a {1 2 3 4 5 6}
```

```
set a [list 1 2 3 4 5 6]  
puts $a
```

built-in function of the list.

1. llength :
2. lindex :
3. lrange :
4. linsert :
5. lreplace:
6. lassign :
7. lappend:
8. lset:
9. split:
10. concat:
11. lsort :
12. lsearch
13. lmap

1. llength :

it is used to find the length of the list.

```
set a [list 1 2 3 4 5 6]  
set b [llength $a]  
puts $b
```

Tcl tool command language

2. lindex :

it is used to access the elements of the list

```
set a [list 1 2 3 4 5 6]
```

```
set b [lindex $a 0]
```

```
puts $b
```

```
set a [list [list 1 2 3 4 5] [list red orange yellow]]
```

```
set b [lindex $a 2 2]
```

```
puts $b
```

3.lrange :

it is used to access the consecutive elements in the list.

```
set x [list red green black yellow]
```

```
set b [lrage $x 0 2]
```

```
puts $b
```

4.linsert :

it is used to insert the element in the list at particular index.

```
set a [list red green black yellow]
```

```
set b [linsert $a 0 white]
```

```
puts $a
```

```
puts $b
```

```
set a [list red green black yellow]
```

```
set a [linsert $a 0 white]
```

```
puts $a
```

5.lreplace:

it is used to replace the elements at particular element.

```
set a [list red green orange]
```

```
set b [lreplace $a 1 1 white]
```

```
puts $b
```

```
set a [list red green orange]
```

```
puts $a
```

Tcl tool command language

```
set b [lreplace $a 0 end white]
puts $b
```

6.lassign :

it is used to assign the elements of the list into variables which we mentioned.

```
set a [list red orange green blue]
lassign $a b c d e
```

```
puts $b
puts $c
puts $d
puts $e
```

7. lappend:

it is used to add/append the element at the end of the existing list.

```
set x [list 1 2 3 4 5 6]
puts $x
lappend x 7
puts $x
```

```
set x [list 1 2 3 4 5 6]
puts $x
lappend x 7 8 9 10 11 12
puts $x
```

8.lset:

it is used to set elements at particular index range.

used to modify the elements in a list.
it allows to change one or more elements in the list.
by specifying the index of the elements
which we need to modify the new value.

syntax:

```
set varname index value
```

```
set a [list 21 22 23 24 25 26]
puts $a
lset a 0 50
puts $a
```

Tcl tool command language

```
set mylist [list a b c d e]
lset mylist 2 C
puts $mylist
```

9.split:
it is used to split the list.

```
set a [list red_green_orange]
puts $a
set b [llength $a]
puts $b
```

```
set c [split $a "_"]
puts $c
puts [llength $c]
```

10. concat:
it is used to join the one or more list.

```
set a [list 1 2 3 4 5]
puts $a
set b [list 6 7 8 9 10]
puts $b
set c [concat $a $b]
puts $c
```

11.lsort :
set all the elements according to ASCII order.

```
set a [list orange Apple BANANNA]
puts $a
set b [lsort $a]
puts $b
```

12. lsearch
The lsearch command in Tcl is used to search for a specific element in a list and return its index.

```
set mylist {apple banana cherry}
set index [lsearch $mylist banana]
puts "Index of banana: $index"
```

13.lmap

Tcl tool command language

The lmap command in Tcl is used to apply a command or script to each element of a list and return a new list containing the results.

```
set mylist {apple banana cherry}
set upperlist [lmap elem $mylist {string toupper $elem}]
puts $upperlist
```

example:1

```
set a {TCL DSD Linux Gvim Custom_layout PD Functional_verification DV DFT
Embedded_system}
puts $a
```

```
set b [lsort $a]
puts $b
```

```
set i 1
foreach c $b {
    puts "course $i is:$c"
    incr i
}
```

#output:

```
course 1 is :Custom_layout
course 2 is :DFT
```

example:2

```
set port1 {p1 p2 p3 p4 p5 p66}
set port2 {p6 p7 p8 p9 p10 p11}
```

```
set port1 {p1 p2 p3 p4 p5 p66 p17}
set port2 {p6 p7 p8 p9 p10 p11 }
```

```
set merged [concat $port1 $port2]
puts $merged
```

```
set len [llength $merged]
puts $len
```

```
set a [expr $len / 4.0]
puts $a
```

```
puts $a
set b [expr ceil($a)]
puts $b
set c [expr int($b)]
```

Tcl tool command language

```
puts $c
if {0} {
#p1  p2  p3  p4  p5  p6  p7  p8  p9  p10
#0   1   2   3   4   5   6   7   8   9
  0   1   $c-1  $c   2*$c-1 2*$c   3*$c-1 3*$c
}
set side1 [lrange $merged 0 [expr $c-1]]
puts "cell side 1 is:$side1"

set side2 [lrange $merged $c [expr 2*$c-1]]
puts "cell side 2 is:$side2"

set side3 [lrange $merged [expr 2*$c] [expr 3*$c-1]]
puts "cell side 3 is:$side3"

set side4 [lrange $merged [expr 3*$c] [expr 4*$c-1]]
puts "cell side 4 is:$side4"
```

example:3

write a tcl program to find the number of tcl is repeated in the a list ?

```
set a {TCL DSD Linux Gvim Custom_layout PD Functional_verification DV DFT
Embedded_system tcl Tcl tcl}
```

```
set b [lsort -nocase $a]
#puts $b
```

```
set d [lmap elem $b {string toupper $elem}]
puts $d
```

```
set i 0
foreach c $d {
if {$c == "TCL"} {
incr i
}
}
```

```
puts "the number of times tcl is repeated is:$i"
```

what are the special variables in TCL?

Tcl tool command language

these are the special variables or predefined variables that hold the important information or values related to the TCL.

1. number of arguments passing to the file.
2. environment variables,
3. version and patch level of TCL being used, information about the platform of TCL.
4. by running on special variables and information about most of recent TCL error.

some of the tcl special variables:

1. \$argc
2. \$argv
3. \$args
4. argv0
5. env
6. tcl_interactive
7. tcl_platform
8. tcl_version
9. tcl_library
10. tcl_patchlevel
11. errorCode
12. errorinfo

1. \$argc - This variable contains the number of command-line arguments passed to a Tcl script.

```
puts $argc
```

2. \$argv - This variable is an array that contains the command-line arguments passed to a Tcl script. The first argument is stored in \$argv(0), the second argument is stored in \$argv(1), and so on.

```
puts $argv
```

3. \$args - This variable is used to capture any additional arguments passed to a Tcl procedure that are not specified in the formal parameter list.

```
proc myproc {x1 x2 args} {  
  puts "x1 value is:$x1"  
  puts "x2 value is:$x2"  
  puts "args value is:$args"  
}
```

Tcl tool command language

```
myproc 1 2 3 4 5 6 7 8
```

4.argv0 : it is used to print the script name.

```
puts $argv0
```

5.env : A dictionary of environment variables and their values.

How to print the env variables

```
puts "Environment variables are:"
```

```
foreach key [array names env] {  
  puts "$key=$env($key)"  
}
```

6. tcl_interactive: A boolean indicating whether TCL is running in interactive mode or not.

7. tcl_platform: A dictionary of information about the platform TCL is running on, including operating system, name, version.

```
puts $tcl_platform(os)  
puts $tcl_platform(osVersion)  
puts $tcl_platform(machine)  
puts $tcl_platform(tclVersion)
```

Some of the commonly used properties of \$tcl_platform are:

os: the name of the operating system

osVersion: the version of the operating system

machine: the name of the hardware platform

user: the name of the user running the Tcl interpreter

8.tcl_version: the version of the Tcl interpreter.

```
puts $tcl_version
```

9.tcl_library: A dictionary where TCL's standard scripts and packages are located.

```
puts $tcl_library
```

10.tcl_patchlevel: the patch level of TCL being used.

```
set tcl_patchlevel [info patchlevel]  
puts $tcl_patchlevel
```


Tcl tool command language

11. errorCode: A string that describes the most recent TCL error.

```
if {[catch {some command} result]} {  
    puts "An error occurred: $result"  
    puts "Error code: $errorCode"  
}
```

12.errorinfo: A list of stack trace information for most recent TCL error.

```
if {[catch {some command} result]} {  
    puts "An error occurred: $result"  
    puts "Error code: $errorCode"  
    puts "Stack trace: $errorInfo"  
}
```

catch an error and print the stack trace and error code

```
if {[catch {expr 1 / 0} result]} {  
    puts "Error: $result"  
    puts "Stack trace: $errorInfo"  
    puts "Error code: $errorCode"  
}
```

example:1

write TCL program for addition of the two numbers using special variables.

```
puts "count of argument is:$argc"  
puts "argument list is:$argv"  
puts "file name is:$argv0"
```

```
if 0 {  
    puts "argument list is:$argv"
```

```
set a [lindex $argv 0]  
set b [lindex $argv 1]
```

```
lassign $argv a b
```

```
set c [expr $a + $b]  
puts "the addition of a and b is:$c"
```

Tcl tool command language

```
}
```

example:2

write TCL program for factorial of a number using special variables.

```
lassign $argv a
```

```
set fact 1
```

```
for {set i 1} {$i <= $a} {incr i} {
```

```
set fact [expr {$fact * $i}]
```

```
}
```

```
puts $fact
```

example : 3

1. calculate the area of the cell using special variables.
passing coordinates and its values using special variables.

2. if we pass the incorrect arguments:
print the error.

3. we need to check $x1 > x2$ and $y1 > y2$.
print the error:

4. all is correct, do the area calculation.

example 1:

```
x1 10 x2 20 y1 20 y2 50
```

area of the cell

example : 3

1. calculate the area of the cell using special variables.
passing coordinates and its values using special variables.

2. if we pass the incorrect arguments:
print the error.

3. we need to check $x1 > x2$ and $y1 > y2$.
print the error:

4. if we pass the coordinates in different order we have calculated area of the cell.
5. all is correct, do the area calculation.

```
lassign $argv a b c d e f g h
```

```
if {$argc != 8} {
```

```
puts "Error: Insufficient arguments: The required arguments is 8"
```

Tcl tool command language

```
} elseif {$b > $d} {  
puts "Error:x1 is greater x2"  
}  
} elseif {$f > $h} {  
puts "Error:y1 is greater y2"  
}  
} else {  
puts "All data is correct:proceed with area calculation"  
}
```

```
if {0} {  
#   a  b   c   d   e   f   g   h  
   y1 10 y2 40 y2 40 x1 60  
  
}
```

```
for {set i 0} {$i <= $argc} {incr i 2} {  
  
switch [lindex $argv $i] {  
"x1" {set x1 [lindex $argv $i+1]}  
"x2" {set x2 [lindex $argv $i+1]}  
"y1" {set y1 [lindex $argv $i+1]}  
"y2" {set y2 [lindex $argv $i+1]}  
}  
}
```

```
set area [expr ($x2-$x1)*($y2-$y1)]  
puts "the area of the cell is:$area"
```

File handling operation :

File handling in Tcl refers to the process of reading from and writing to files using Tcl commands. Tcl provides a set of commands to open, read, write, and close files. The basic steps for file handling in Tcl are as follows:

Open a file using the "open" command. This command takes two arguments: the name of the file to be opened and the mode in which the file should be opened (read, write, or append).

```
set read [open "new.txt" r]
```

Read from or write to the file using the appropriate commands. The "gets" command is used to read a single line from a file, while the "read" command can be used to read a specified number of bytes. The "puts" command is used to write a string to a file.

Close the file using the "close" command when finished.

r: This mode opens the file for reading. If the file doesn't exist, an error is raised.

Tcl tool command language

```
set file [open "new.txt" r]
```

```
# how to read the single line of the file
```

```
set data [gets $file]
```

```
puts $data
```

```
# Read entire content of the file
```

```
set data [read $file]
```

```
puts $data
```

```
# it reads the line by line of the file.
```

```
while {[gets $file line] >= 0} {
```

```
puts $line
```

```
}
```

```
close $file
```

w: This mode opens the file for writing, but if the file already exists, its contents are truncated (i.e., deleted). If the file doesn't exist, a new file is created.

```
set write [open "example.txt" "w"]
```

```
puts $write "welcome to vlsiguru"
```

```
close $write
```

a: This mode opens the file for writing, but if the file already exists, new data is appended to the end of the file. If the file doesn't exist, a new file is created.

```
set write [open "example.txt" "a"]
```

```
puts $write "welcome to bangalore"
```

```
close $write
```

r+: This mode opens the file for reading and writing, starting at the beginning of the file. If the file doesn't exist, an error is raised.

w+: This mode opens the file for reading and writing, but if the file already exists, its contents are truncated (i.e., deleted). If the file doesn't exist, a new file is created.

a+: This mode opens the file for reading and writing, but if the file already exists, new data is appended to the end of the file. If the file doesn't exist, a new file is created.

how to read file:

Tcl tool command language

```
set file [open "example.txt" "r"]
set data [gets $file]
close $file
```

how to write a file:

```
set file [open "example.txt" "w"]
puts $file "Hello, World!"
close $file
```

how to append the file:

```
set file [open "example.txt" "a"]
puts $file "welcome to vlsiguru"
close $file
```

what is the use of file handling in tcl ?

Reading and writing data to and from files: One of the primary uses of file handling in Tcl is to read data from files and write data to files. This is useful for working with large datasets, log files, configuration files, and other types of files.

Parsing data from files: Tcl can be used to extract data from files using regular expressions, string manipulation, and other techniques. This is useful for working with structured data such as CSV files, XML files, and JSON files.

Creating and modifying files: Tcl can be used to create new files or modify existing files. This is useful for generating reports, creating backups, and other file-based operations.

Interacting with external programs: Tcl can execute external programs and interact with them by redirecting input and output to files. This is useful for automating tasks and integrating with other systems.

list of file types supported by Tcl:

- Text files
- Binary files
- CSV files
- XML files
- JSON files
- Database files (SQLite, MySQL, PostgreSQL)
- INI files
- Configuration files
- Log files

Tcl tool command language

Script files
HTML files
PDF files
Image files (JPEG, PNG, GIF, BMP, etc.)
Audio files (MP3, WAV, etc.)
Video files (MP4, AVI, etc.)
Compressed files (ZIP, TAR, GZIP, etc.)

example:1

Read the new.txt file and display all the lines
and calculate the area of each cell and
total area of all the cells using file
handling operation.

```
set file [open "new.txt" r]
set file1 [open "area_detail.txt" a]
while {[gets $file line] >= 0} {
    lassign $line a b c d e f g h i
    set area [expr ($e-$c)*($i-$g)]
    puts "Area of the $a is:$area"
    puts $file1 "Area of the $a is:$area"
}
close $file
```

example:2

Read the new.txt file and display all the lines
and calculate the area of each cell and
total area of all the cells using file
handling operation.(note: if coordinates are in not order, if check the arguments value, if
arguments are not sufficient, print error message, $x_1 > x_2$ and $y_1 > y_2$, print error)

example:3

pass the line number as a argument.calculate the area of the
particular line.

example:4

pass the argument parameter of the cell name of the line.
find the area of the cell using filehandling operation

example:2:

Read the new.txt file and display all the lines
and calculate the area of each cell and

Tcl tool command language

total area of all the cells using file handling operation.(if coordinates are in not order)

```
set fh_read [open "new.txt" r]
set fh_write [open "new_cell.txt" w]
set total_area 0
while {[gets $fh_read line] >= 0} {
    set a [llength $line]
    if {$a == 0} {
        continue
    }
    for {set i 1} {$i < $a} {incr i 2} {
        switch [lindex $line $i] {
            "X1" {set X1 [lindex $line $i+1]}
            "X2" {set X2 [lindex $line $i+1]}
            "Y1" {set Y1 [lindex $line $i+1]}
            "Y2" {set Y2 [lindex $line $i+1]}
        }
    }
    set area [expr ($X2 - $X1)*($Y2 - $Y1)]
    puts "the area of the cell [lindex $line 0] is:$area"
    puts $fh_write "the area of the cell [lindex $line 0] is:$area"
    set total_area [expr $area+$total_area]
}

puts "the total area of all cell is:$total_area"
puts $fh_write "the total area of all cell is:$total_area"
```

example:3

pass the line number as a argument.calculate the area of the particular line.

```
lassign $argv ln
set fh_read [open "new.txt" r]
set line_number 0
while {[gets $fh_read line] >= 0} {
    set a [llength $line]
    if {$a == 0} {
        continue
    }
    incr line_number
    if {$line_number == $ln} {
        for {set i 1} {$i < $a} {incr i 2} {
            switch [lindex $line $i] {
```

Tcl tool command language

```
"X1" {set X1 [lindex $line $i+1]}
"X2" {set X2 [lindex $line $i+1]}
"Y1" {set Y1 [lindex $line $i+1]}
"Y2" {set Y2 [lindex $line $i+1]}
}
}
set area [expr ($X2 - $X1)*($Y2 - $Y1)]
puts "the area of the cell [lindex $line 0] is:$area"
}
}
if {$ln > $line_number || $ln <= 0} {
puts "Error: line number not found"
}
```

example:1

pass the argument parameter of the cell name of the line.

find the area of the cell using filehandling operation.

lassign \$argv para

set fh_read [open "new.txt" r]

set flag 0

while {[gets \$fh_read line] >= 0} {

set l [lindex \$line]

if {\$l == 0} {

continue

}

for {set i 1} {\$i < \$l} {incr i 2} {

switch [lindex \$line \$i] {

"X1" {set X1 [lindex \$line \$i+1]}

"X2" {set X2 [lindex \$line \$i+1]}

"Y1" {set Y1 [lindex \$line \$i+1]}

"Y2" {set Y2 [lindex \$line \$i+1]}

}

}

if {\$para == [lindex \$line 0]} {

set flag 1

set area [expr (\$X2-\$X1)*(\$Y2-\$Y1)]

puts "the area of the \$para is:\$area"

}

}

if {\$flag == 0} {

puts "Error:Wrong cell name"

exit

}

Example:2

Based on inputs passed to as argument.find

Tcl tool command language

the corresponding parameter value of each cell in the table.txt file using file handling operation.

```
lassign $argv a b
set fh_read [open "table.txt" r]

set flag 0
while {[gets $fh_read line] >= 0} {
set c [lindex $line 0]
if {$a == $c} {
    if {$b == "MRT"} {
        set d [lindex $line 1]
    } elseif {$b == "MFT"} {
        set d [lindex $line 2]
    } elseif {$b == "MIRT"} {
        set d [lindex $line 3]
    } elseif {$b == "MIFT"} {
        set d [lindex $line 4]
    } elseif {$b == "scenario"} {
        set d [lindex $line 5]
    } else {
        puts "Error:wrong argument parameter"
        exit
    }
set flag 1
puts "the corresponding value of $a and $b is:$d"
}
}
if {$flag == 0} {
puts "Error:wrong argument with CLK name"
}
```

example:3

find the number of times 0.10 is present in the table.txt file.

```
set fh_read [open "table.txt" r]

set i 0
while {[gets $fh_read line] >= 0} {
foreach a $line {
if {$a == 0.10} {
incr i
}
}
}
```

Tcl tool command language

```
}  
puts "the number of times 0.10 is present is:$i"
```

example:4

find the number of times the value is present in the table.txt file.
pass the value as argument.

```
lassign $argv value  
set fh_read [open "table.txt" r]  
  
set i 0  
while {[gets $fh_read line] >= 0} {  
  foreach a $line {  
    if {$a == $value} {  
      incr i  
    }  
  }  
}  
if {$i == 0} {  
  puts "Error:passed value not found in the file"  
} else {  
  puts "the number of times $value is present is:$i"  
}
```

what is procedure in the tcl?

it is similar to function or subroutine with respect to
other prog lang.

what is procedure in TCL?

In TCL (Tool Command Language), a procedure is a reusable code block that can be called multiple times with different inputs. It is similar to a function or subroutine in other programming languages.

how to define the procedure in the tcl ?

```
proc proc_name {arguments} {
```

body of the proc

Tcl tool command language

```
}
```

proc_name

1. write a proc without arguments:

```
proc show {} {  
  puts "hello world"  
}
```

show

```
proc show {} {  
  set a "hello world"  
  puts $a  
}
```

show

2. write a proc with 2 arguments:

```
proc add {a b} {  
  set c [expr $a + $b]  
  puts $c  
}
```

add 10 20

3. write a proc with arguments ?

```
proc proc_name {arg1 arg2 arg3} {  
  
  set c [expr $arg1 + $arg2 + $arg3]  
  puts $c  
}
```

proc_name 1 2 3

4. write a proc with return keyword in the procedure ?

```
proc add {x1 x2} {  
  set c [expr $x1 + $x2]  
  return $c  
}
```

Tcl tool command language

```
puts "Addition of x1 and x2 is:[add 5 10]"
```

5. write a proc for addition, sub, mul and div of the two numbers.

```
proc arith {x1 x2 a b} {  
  if {$x1 == 0 && $x2 == 0} {  
    set c [expr $a + $b]  
    return $c  
  } elseif {$x1 == 0 && $x2 == 1} {  
    set c [expr $a - $b]  
    return $c  
  } elseif {$x1 == 1 && $x2 == 0} {  
    set c [expr $a * $b]  
    return $c  
  } elseif {$x1 == 1 && $x2 == 1} {  
    set c [expr $a / $b]  
    return $c  
  }  
}  
puts "Addition of a and b is:[arith 0 0 10.0 5]"  
puts "Sub of a and b is:[arith 0 1 10.0 5]"  
puts "Mul of a and b is:[arith 1 0 10.0 5]"  
puts "Div of a and b is:[arith 1 1 10.0 5]"
```

output:

```
x1=0 and x2=0  addition  
x1=0 and x2=1  sub  
x1=1 and x2=0  mux  
x1=1 and x2=1  div
```

6. write a tcl pgm to print factorial of the number using proc?

```
proc fac {a} {  
  set b 1  
  for {set i 1} {$i <= $a} {incr i} {  
    set b [expr $i*$b]  
  }  
  return $b  
}  
puts [fac 4]
```

or

Tcl tool command language

```
proc fact { num } {  
    set result 1  
    while { $num >=1} {  
        set result [expr $result * $num]  
        incr num -1  
    }  
    puts $result  
    return $result  
}  
fact 0
```

7. write a tcl pgm to print all the prime numbers from 2 to 100 using proc?

Assignmnet.

8. write a tcl pgm to find the area of the cell, pass the file and line number as argument value using proc.

```
proc area {file_name line_number} {  
    set file [open "$file_name" r]  
    set ln 0  
    while {[gets $file line] >= 0} {  
        set length [llength $line]  
        if {$length == 0} {  
            continue  
        }  
        incr ln  
        if {$line_number == $ln} {  
            for {set i 1} {$i < $length} {incr i 2} {  
                switch [lindex $line $i] {  
                    "x1" {set x1 [lindex $line $i+1]}  
                    "x2" {set x2 [lindex $line $i+1]}  
                    "y1" {set y1 [lindex $line $i+1]}  
                    "y2" {set y2 [lindex $line $i+1]}  
                }  
            }  
            set area [expr ($x2-$x1)*($y2-$y1)]  
            puts "Area of the [lindex $line 0] is:$area"  
        }  
        if {$line_number > $ln || $line_number == 0} {  
            puts "Error:Wrong line number"  
        }  
    }  
}
```

Tcl tool command language

area new.txt 0

9.write a tcl pgm to find the area of the cell, pass the file and cell namer as argument value using proc.

```
proc area {file_name parameter} {
    set file [open $file_name r]
    set flag 0
    while {[gets $file line] >= 0} {
        set length [llength $line]
        if {$length == 0} {
            continue
        }
        set para [lindex $line 0]
        if {$parameter == $para} {
            for {set i 1} {$i < $length} {incr i 2} {
                switch [lindex $line $i] {
                    "x1" {set x1 [lindex $line $i+1]}
                    "x2" {set x2 [lindex $line $i+1]}
                    "y1" {set y1 [lindex $line $i+1]}
                    "y2" {set y2 [lindex $line $i+1]}
                }
            }
            set flag 1
            set area [expr ($x2-$x1)*($y2-$y1)]
            puts "Area of the [lindex $line 0] is:$area"
        }
    }
    if {$flag == 0} {
        puts "Error:Wrong cell name"
    }
}
area new.txt SRAM
```

10.Based on inputs passed to as argument.find the corrsponding parameter value of each cell in the table.txt file using proc.

```
proc value {file_name cell_name para} {
    set file [open $file_name r]
    while {[gets $file line] >= 0} {
        set c [lindex $line 0]
        if {$cell_name == $c} {
            if {$para == "MRT"} {
                set d [lindex $line 1]
            } elseif {$para == "MFT"} {
```

Tcl tool command language

```
    set d [lindex $line 2]
  } elseif {$para == "MIRT"} {
    set d [lindex $line 3]
  } elseif {$para == "MIFT"} {
    set d [lindex $line 4]
  } elseif {$para == "scenario"} {
    set d [lindex $line 5]
  } else {
    puts "Error:wrong parameter"
    exit
  }
}
}
puts "the cell name is $cell_name and parameter is $para it's value is :$d"
}
value table.txt PCI_CLK1 MFT
```

11.find the number of times 0.10 is present in the table.txt file using proc.

```
proc count {file_name} {
  set file [open $file_name r]
  set count 0
  while {[gets $file line] >= 0} {
    foreach b $line {
      if {$b == 0.10} {
        incr count
      }
    }
  }
  puts "the number of times 0.10 is present in the file is:$count"
}
count table.txt
```

12. find the number of times the value is present in the table.txt file using proc ?

```
proc count {file_name value} {
  set file [open $file_name r]
  set count 0
  while {[gets $file line] >= 0} {
    foreach b $line {
      if {$b == $value} {
        incr count
      }
    }
  }
}
```

Tcl tool command language

```
}  
}  
puts "the number of times $value is present in the file is:$count"  
}
```

count table.txt 0.25

```
proc myProcedure {arg1 arg2} {  
    # code to be executed  
}
```

myProcedure value1 value2

Procedures can be used to simplify code and make it more modular and reusable. They are an important part of the TCL language and are used extensively in TCL programming.

what is the uses of the proc in tcl ?

1. Modular programming: Procedures allow you to break up your code into smaller, more manageable pieces. This makes your code easier to read, understand, and maintain.
2. Code reusability: Once you define a procedure, you can call it multiple times from different parts of your code, without having to write the same code again and again. This saves time and reduces the risk of errors.
3. Parameterization: Procedures can take input parameters, which allows you to pass in different values and customize the behavior of the procedure based on those values. This makes your code more flexible and adaptable.
4. Encapsulation: Procedures can be used to encapsulate complex logic or algorithms, making it easier to reuse or share that logic with other developers.

write a procedure without the arguments:

```
proc show {} {  
    puts "hello world!"  
}
```

show

write proc with return

Tcl tool command language

```
proc show {} {  
  set a "welcome to bangalore"  
  return $a  
}
```

```
puts [show]
```

write a proc with variable number of the arguments ?

```
proc sum {args} {  
  set s 0  
  foreach arg $args {  
    incr s $arg ; #s + 1  
  }  
  return $s  
}
```

```
puts [sum 1 2 3 4 5]  
puts [sum 1 2 3 4]  
puts [sum 1 2 3]  
puts [sum 1 2]  
puts [sum 1]
```

Arguments with default values/implicit arguments.

Implicit arguments in TCL procedures may have implicit values.

An Implicit arguments value is used if no explicit value is provided.

```
proc show {a {b 2}} {  
  puts $a  
  puts $b  
}
```

```
show 10 20
```

```
proc add {a {b 2}} {  
  set c [expr $a + $b]  
  return $c  
}
```

```
puts [add 2]
```

how to return mutiple values in the proc.

#the return command passes one value to caller.

Tcl tool command language

there is often a need to return the multiple values. in such cases we can make return list.

```
proc arti {a b} {  
  set m1 [expr $a + $b]  
  set m2 [expr $a - $b]  
  set m3 [expr $a * $b]  
  set m4 [expr $a / $b]  
  return [list $m1 $m2 $m3 $m4]  
}
```

```
puts [arti 20.0 10]
```

scope of the variables :

local variables:

A variable declared inside a procedure has a procedure scope/local variable.

```
proc show {} {  
  set a 10  
  puts $a  
}
```

```
puts $a  
show
```

global variables:

Any variable which is declared outside the proc. we can also use this variable inside the proc.

```
set b 20
```

```
proc scope {a} {  
  global b  
  set c [expr $a + $b]  
  return $c  
}
```

```
puts [scope 10]
```

```
set d [expr $b + $b]  
puts $d
```

Tcl tool command language

Recurison in the proc:

Recursion is way to call the procedure itself repeatedly.

Recurison is a way of declearing proc in which the proc being defined is applied within its own definition.

In other word, a recurison function calls itself to do its job.

```
proc sumnum {a} {  
  if {$a > 0} {  
    return [expr $a + [sumnum [expr $a - 1]]]  
  } else {  
    return 0  
  }  
}
```

```
puts [sumnum 5]
```

example:1

write a TCL code for arithmetic operation using proc.

```
proc arti {X1 X2 a b} {  
  
  if {$x1 == 0 && $x2 == 0} {  
    return [expr $a + $b]  
  } elseif {$x1 == 0 && $x2 == 1} {  
    return [expr $a - $b]  
  } elseif {$x1 == 1 && $x2 == 0} {  
    return [expr $a * $b]  
  } elseif {$x1 == 1 && $x2 == 1} {  
    return [expr $a / $b]  
  }  
  
}
```

```
set a 20.0  
set b 10
```

```
puts "x1=0 and x2=0 is add:[arti 0 0 $a $b]"  
puts "x1=0 and x2=1 is sub:[arti 0 1 $a $b]"  
puts "x1=1 and x2=0 is mul:[arti 1 0 $a $b]"  
puts "x1=1 and x2=1 is div:[arti 1 1 $a $b]"
```

Tcl tool command language

example:2

find the area of the particular line of the file new.txt using proc.
puts [area new.txt 1]

```
proc area {x y} {  
  
    set fh_read [open $x r]  
    set line_number 0  
    while {[gets $fh_read line] >= 0} {  
        set a [llength $line]  
        if {$a == 0} {  
            continue  
        }  
        incr line_number  
        if {$line_number == $y} {  
            for {set i 1} {$i < $a} {incr i 2} {  
                switch [lindex $line $i] {  
                    "X1" {set X1 [lindex $line $i+1]}  
                    "X2" {set X2 [lindex $line $i+1]}  
                    "Y1" {set Y1 [lindex $line $i+1]}  
                    "Y2" {set Y2 [lindex $line $i+1]}  
                }  
            }  
            set area [expr ($X2 - $X1)*($Y2 - $Y1)]  
            puts "the area of the cell [lindex $line 0] is:$area"  
        }  
        if {$y > $line_number || $y <= 0} {  
            puts "Error: line number not found"  
        }  
    }  
    area new.txt 0  
}
```

example:3

find the area of the particular line of the file new.txt using proc.
puts [area new.txt SRAM]

```
proc area {fn cn} {  
  
    set fh_read [open $fn r]  
    set flag 0  
    while {[gets $fh_read line] >= 0} {  
        set l [llength $line]  
        if {$l == 0} {
```

Tcl tool command language

```
continue
}
for {set i 1} {$i < $l} {incr i 2} {
switch [lindex $line $i] {
"X1" {set X1 [lindex $line $i+1]}
"X2" {set X2 [lindex $line $i+1]}
"Y1" {set Y1 [lindex $line $i+1]}
"Y2" {set Y2 [lindex $line $i+1]}
}
}
if {$scn == [lindex $line 0]} {
set flag 1
set area [expr ($X2-$X1)*($Y2-$Y1)]
puts "the area of the $scn is:$area"
}
}
if {$flag == 0} {
puts "Error:Wrong cell name"
exit
}
}
```

area new.txt PLL

example:4

Write a TCL program to find the corresponding value of the cell name and parameter of the table.txt using proc.

```
proc paravalue {fn a b} {
set fh_read [open $fn r]

set flag 0
while {[gets $fh_read line] >= 0} {
set c [lindex $line 0]
if {$a == $c} {
if {$b == "MRT"} {
set d [lindex $line 1]
} elseif {$b == "MFT"} {
set d [lindex $line 2]
} elseif {$b == "MIRT"} {
set d [lindex $line 3]
} elseif {$b == "MIFT"} {
set d [lindex $line 4]
} elseif {$b == "scenario"} {
set d [lindex $line 5]
}
```

Tcl tool command language

```
} else {  
  puts "Error:wrong argument parameter"
```

Procedures can be used to simplify code and make it more modular and reusable. They are an important part of the TCL language and are used extensively in TCL programming.

What are the uses of the proc in tcl ?

1. Modular programming: Procedures allow you to break up your code into smaller, more manageable pieces. This makes your code easier to read, understand, and maintain.
2. Code reusability: Once you define a procedure, you can call it multiple times from different parts of your code, without having to write the same code again and again. This saves time and reduces the risk of errors.
3. Parameterization: Procedures can take input parameters, which allows you to pass in different values and customize the behavior of the procedure based on those values. This makes your code more flexible and adaptable.
4. Encapsulation: Procedures can be used to encapsulate complex logic or algorithms, making it easier to reuse or share that logic with other developers.

Write a procedure without the arguments:

```
proc show {} {  
  puts "hello world!"  
}
```

show

Write a procedure with return

```
proc show {} {  
  set a "welcome to bangalore"  
  return $a  
}
```

puts [show]

Write a procedure with variable number of arguments ?

```
proc sum {args} {  
  set s 0
```

Tcl tool command language

```
foreach arg $args {  
  incr s $arg ; #s + 1  
}  
return $s  
}
```

```
puts [sum 1 2 3 4 5]  
puts [sum 1 2 3 4]  
puts [sum 1 2 3]  
puts [sum 1 2]  
puts [sum 1]
```

Arguments with default values/implicit arguments.
Implicit arguments in TCL procedures may have implicit values.
An Implicit arguments value is used if no explicit value is provided.

```
proc show {a {b 2}} {  
  puts $a  
  puts $b  
}
```

```
show 10 20
```

```
proc add {a {b 2}} {  
  set c [expr $a + $b]  
  return $c  
}
```

```
puts [add 2]
```

how to return multiple values in the proc.
#the return command passes one value to caller.
there is often a need to return the multiple values. in such cases we can make return list.

```
proc arti {a b} {  
  set m1 [expr $a + $b]  
  set m2 [expr $a - $b]  
  set m3 [expr $a * $b]  
  set m4 [expr $a / $b]  
  return [list $m1 $m2 $m3 $m4]  
}
```

```
puts [arti 20.0 10]
```

Tcl tool command language

scope of the variables in the proc:

local variables:

A variable declared inside a procedure has a procedure scope/local variable.

```
proc show {} {  
  set a 10  
  puts $a  
}
```

```
puts $a  
show
```

global variables:

Any variable which is declared outside the proc. we can also use this variable inside the proc.

```
set b 20
```

```
set b 20
```

```
proc scope {a} {  
  global b  
  set c [expr $a + $b]  
  return $c  
}
```

```
puts "using global variable inside the proc:[scope 10]"
```

```
#set d [expr $b + $b]  
#puts "global variable is used outside the proc:$d"
```

Recursion in the proc:

Recursion is a way to call the procedure itself repeatedly.

Recursion is a way of declaring proc in which the proc being defined is applied within its own definition.

In other words, a recursion function calls itself to do its job.

```
proc sumnum {a} {  
  if {$a > 0} {
```


Tcl tool command language

```
return [expr $a + [sumnum [expr $a - 1]]]
} else {
return 0
}
}
```

```
puts [sumnum 5]
```

```
proc fac {a} {
if {$a > 0} {
return [expr $a * [fac [expr $a - 1]]]
} else {
return 1
}
}
```

```
puts "The factorial of the given number is:[fac 6]"
```

In Tcl, a "nested proc" is a procedure that is defined inside another procedure. This means that the nested procedure is only visible and accessible within the scope of the outer procedure. Once the outer procedure returns or is exited, the nested procedure is no longer accessible.

```
proc outer_proc {} {
    proc inner_proc {name} {
        puts "Hello, $name!"
    }
    inner_proc "Bob"
}
```

```
outer_proc
```

what is array in tcl ?

In Tcl, an array is a data structure that stores a collection of values, which can be accessed using an index. It is similar to a list or a dictionary in other programming languages. An array is defined using the "array" command, followed by the name of the array and its values.

```
array set numbers { 1 10 2 20 3 30}
```

```
set first_number $numbers(1)
puts $first_number
```

Tcl tool command language

```
set second_number $numbers(2)
puts $second_number
set third_number $numbers(3)
puts $third_number
```

Array in TCL:

In TCL array is a data structure that stores a collection of values under single variable name.

Array is systematic arrangements of elements with the indices.

Each value in the array is associated with an index which is a unique identifier.

the value in TCL array can be different types, such as integers, strings or other arrays.

the following are some of the operators that can be used in TCL arrays.

1. array set : creates or update an array with a list of values.
2. array get : retrieves values from an array and return them as a list.
3. array size: it returns the number of elements in an array.
4. array names : it returns the names(indices) of all the elements.
5. array exist : it returns 1 if array is exist else 0 otherwise.
6. array unset : removes an array and all its elements.

parray : is used to print the array.

syntax to create the array:

```
# create the array
array set colour {0 black 1 white 2 blue 3 orange 4 red}
```

Tcl tool command language

```
#prints the entire array  
#parray colour
```

```
#prints the 0 indices array.  
puts $colour(0)
```

```
array set colour {0 black 1 white 2 blue 3 orange 4 red}  
#Retrieve all the values in the array as a list  
parray colour
```

```
set myarray [array get colour]  
puts $myarray
```

```
# get the number of elements in the array  
array set colour {0 black 1 white 2 blue 3 orange 4 red}  
set myarray [array size colour]  
puts $myarray
```

```
# check if array is exists  
array set colour {0 black 1 white 2 blue 3 orange 4 red}
```

```
if {[array exist colour]} {  
  puts "Array is exist"  
} else {  
  puts "Array is not exist"  
}
```

```
# how to delete or uninitailize the array  
array set colour {0 black 1 white 2 blue 3 orange 4 red}
```

```
array unset colour
```

```
if {[array exist colour]} {  
  puts "Array is exist"  
} else {  
  puts "Array is not exist"  
}
```

```
# its prints all the indices of the array  
array set colour {a black b white 2 blue 3 orange 4 red}
```

Tcl tool command language

```
set myarray [array names colour]
puts $myarray
```

```
# to get only the values of the array
array set colour {0 black 1 white 2 blue 3 orange 4 red}
```

```
foreach item [array names colour] {

puts $colour($item)

}
```

In TCL, all arrays by nature are associative.
Arrays are stored and retrieved without any specific order.

Associative arrays have an index that is not necessarily a number.

the conventional way to create the array:

```
array set colour {0 red 1 white 3 black 4 green }
```

```
array set colour {0 red 1 white 3 black 4 green }
#Before add the array element
parray colour
```

```
set colour(2) orange
#after adding the array element
parray colour
```

```
array set colour {0 red 1 white 3 black 4 green }
parray colour
```

```
set colour(0) orange
parray colour
```

```
array set colour {0 red 1 white 3 black 4 green }
parray colour
set colour(0) orange
parray colour
```

how to create the array individually:

```
set colour(0) red
set colour(1) white
set colour(2) black
parray colour
```

Tcl tool command language

#how to delete particular element or unitalize
particular element array.

```
set colour(0) red
set colour(1) white
set colour(2) black
#parray colour
```

```
unset colour(1)
parray colour
```

Example 1: Numeric array

```
# Define an array of integers
array set numbers {
    1 10
    2 20
    3 30
}
```

```
# Access elements of the array
set num1 $numbers(1) ;# num1 is 10
set num2 $numbers(2) ;# num2 is 20
set num3 $numbers(3) ;# num3 is 30
```

```
# Loop through all the elements of the array
foreach index [array names numbers] {
    set value $numbers($index)
    puts "numbers($index) = $value"
}
```

Example 2: Associative array# Define an associative array

```
array set person {
    name "John Smith"
    age 30
    city "New York"
}
```

```
# Access elements of the array
set name $person(name) ;# name is "John Smith"
set age $person(age) ;# age is 30
set city $person(city) ;# city is "New York"
```

```
# Loop through all the elements of the array
foreach key [array names person] {
```

Tcl tool command language

```
    set value $person($key)
    puts "$key = $value"
}
```

Example 3: Multi-dimensional array

Define a 2D array of integers

```
array set matrix {
    {0,0} 1
    {0,1} 2
    {1,0} 3
    {1,1} 4
}
```

Access elements of the array

```
set m00 $matrix(0,0) ;# m00 is 1
set m01 $matrix(0,1) ;# m01 is 2
set m10 $matrix(1,0) ;# m10 is 3
set m11 $matrix(1,1) ;# m11 is 4
```

Loop through all the elements of the array

```
foreach indices [array names matrix] {
    set value $matrix($indices)
    puts "matrix($indices) = $value"
}
```

example:1

```
./file.tcl 1
```

output: day is monday

example:2

```
./file.tcl std_name sub
```

1. using the arguments we want to read student name and subject.
2. if we mentioned any student name and subject name.
3. based on the student name and subject name we have get the marks particular subject.
4. print the error if student name and subject name is not there in database.Array in TCL:

Tcl tool command language

example:1
tclsh p.tcl 2

output: day is monday

lassign \$argv day

```
array set days {  
  1 monday  
  2 tuesday  
  3 wednesday  
  4 thursday  
  5 friday  
  6 saturday  
  7 sunday  
}
```

```
foreach key [array names days] {  
  if {$day == $key} {  
    puts "the day $day is:$days($key)"  
  } elseif {$day == 0 || $day > 7} {  
    puts "Error:day is not match"  
    exit  
  }  
}
```

example:2
tclsh p.tcl std_name sub

1. using the arguments we want to read student name and subject.
2. if we mentioned any student name and subject name.
3. based on the student name and subject name we have get the marks for the particular subject.
4. print the error if student name and subject name is not there in database.

Array in tcl ?

array operators in the last session ?

example:2
tclsh p.tcl std_name sub

1. using the arguments we want to read student name and subject.
2. if we mentioned any student name and subject name.
3. based on the student name and subject name we have get the

Tcl tool command language

marks particular subject.

4. print the error if student name and subject name is not there in database.

```
lassign $argv stdn subn
      #std1 p
set sub {p c m}
array set student {
std1 {10 15 20}
std2 {20 30 40}
std3 {30 40 50}
std4 {50 60 70}
}
```

```
set flag 0
foreach a [array names student] {
if {$stdn == $a} {
set flag 1
}
}
if {$flag == 0} {
puts "Error: the student name is not found in the database"
exit
}
```

```
set flag 0
foreach b $sub {
if {$subn == $b} {
set flag 1
}
}
```

```
if {$flag == 0} {
puts "Error: the subject name is not found in the database"
exit
}
```

```
foreach c [array names student] {
if {$stdn == $c} {
if {$subn == "p"} {
set d [lindex $student($c) 0]
} elseif {$subn == "c"} {
set d [lindex $student($c) 1]
} elseif {$subn == "m"} {
set d [lindex $student($c) 2]
```


Tcl tool command language

```
}  
}  
}
```

```
puts "student_name:$stdn subject_name:$subn Marks:$d"
```

Another data type of tcl is mapping data type:

Dictionary :

In Tcl, a dictionary is a data structure that associates keys with values. It is similar to an array, but unlike an array, the keys can be any Tcl value, not just integers or strings.

Dictionary:

In TCL, Dic essentially a associative array with the keys and values pair.

In TCL, a dic is a data structure that stores key-value pair. where keys are unique.

In TCL, A Dic is an arragment for mapping values to keys.

Dic are created using arrays.

Dic are faster than the arrays.

how to create the dic.

how to create the dic is

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue"]  
puts $colour
```

#to access the elements of a dic, you can use \$ operator.

#how to add the values in to the dic

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue"]  
pust $colour
```

```
dict set $colour colour4 "orange"  
puts $colour
```

Tcl tool command language

dict create : is used to create the dic.

#how to add the values in to the dic

set a value to the dic.

dict set: to add the value to dic

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue"]  
puts $colour
```

```
dict set colour colour4 "orange"  
puts $colour
```

dict get: retrieves the value of the specific key in the dict.

#get the value of the key.

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue" colour4 "orange"]  
puts $colour
```

```
set val [dict get $colour colour2]  
puts $val
```

#how to find out the size of the dict.

dict size :Returns the number of key-value pair in the dic.

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue" colour4 "orange"]  
puts $colour
```

```
puts "size of the dict:[dict size $colour]"
```

dict exists: Returns a boolean indicating whether a specific key exists in the dict.

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue" colour4 "orange"]  
puts $colour
```

```
if {[dict exists $colour colour5]} {  
    puts "colour5 is exists in the dict"  
} else {  
    puts "this key is not exists in the dict"  
}
```

```
set a [dict exists $colour colour1]  
puts $a
```

Tcl tool command language

dict keys:Returns a list of all the keys in the dict.

get the keys of the dict.

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue" colour4 "orange"]
puts $colour
```

```
puts "the keys of the dic is:[dict keys $colour]"
```

dict values: Returns a list of all the values in the dict.

#get all the values of the dict.

```
set colour [dict create colour1 "black" colour2 "white" colour3 "blue" colour4 "orange"]
puts $colour
```

```
puts "the values of the dic is:[dict values $colour]"
```

dict unset: Removes a specific key from the dict.

remove a key-value pair from the dict.

```
set colour [dict create colour1 {"black" "purpple" } colour2 "white" colour3 "blue" colour4
"orange"]
puts $colour
```

```
dict unset colour colour1
```

```
puts $colour
```

dict append:add at the end of the dict

```
set colour [dict create colour1 {"black" "purpple" } colour2 "white" colour3 "blue" colour4
"orange"]
puts $colour
```

```
dict append colour colour5 "yellow"
```

```
puts $colour
```

dict operators:

1. dict create
2. dict set
3. dict get
4. dict size
5. dict exists

Tcl tool command language

- 6. dict keys
- 7. dict values
- 8. dict unset
- 9. dict append

Dict iteration:

```
set colour [dict create colour1 {black yellow} colour2 "white" colour3 "blue" colour4 "orange"]  
puts $colour
```

```
puts [dict keys $colour]  
puts [dict values $colour]
```

```
foreach item [dict keys $colour] {  
  set val [dict get $colour $item]  
  puts $val  
}
```

example 2 in the array repeat the same using dict and proc.

using proc and dict :

using dict create the database:

student marks database:

```
student1 {10 11 12 13}  
student2 {20 21 22 23}  
student3 {30 31 32 33}  
student4 {40 41 42 43}  
student5 {50 51 52 53}
```

in the proc if we passing the arguments are [student1 4]

output:

```
student name is :student1  
student sub is:1  
marks is:10
```

```
proc student_marks {std_name sub_number} {
```

```
  set class_marks [dict create student1 {10 11 12 13 14} student2 {20 21 22 23 14}  
  student3 {30 31 32 33 14} student4 {40 41 42 43 14} student5 {50 51 52 53 14}]
```

Tcl tool command language

```
foreach a [dict keys $class_marks] {  
  if {$std_name == $a} {  
    set b [dict get $class_marks $a]  
    set c [lindex $b $sub_number-1]  
    puts "student name is:$std_name subject number is:$sub_number Marks is:$c"  
  }  
}  
}
```

student_marks student5 5

Regular expression in TCL:

it is used to match the pattern in the text.

In TCL, regular expression is pattern that describes a set of strings, extract the information, and perform complex text processing tasks.

regex command is supporting to match the pattern.

this command allows to perform various operations such as matching, substitution and splitting on the strings using regular expression.

syntax:

```
[regex "search_pattern" $content]
```

Regular expressions are a powerful tool for pattern matching and string manipulation in Tcl. In physical design and dft, regular expressions can be useful in several ways:

Parsing Design Files: Regular expressions can be used to extract useful information from design files such as LEF, DEF, GDSII, and Verilog. For example, you could use regular expressions to extract the coordinates of specific design elements or to extract pin names and locations.

Netlist Manipulation: Regular expressions can also be used to manipulate netlists. You could use regular expressions to extract the names of all the nets that connect to a particular pin or to extract the names of all the pins connected to a particular net.

Scripting Automation: Regular expressions can help automate certain tasks in Tcl scripts. For example, you could use regular expressions to search and replace text in a file, or to extract certain patterns of text from a log file.

Overall, regular expressions are a powerful tool for pattern matching and string manipulation in Tcl. They can be used to extract useful information from design files,

Tcl tool command language

manipulate netlists, and automate certain tasks in Tcl scripts. By understanding regular expressions, you can become more effective in physical design and scripting tasks.

example:4

1. Read the VHDL.txt file
2. Read line by line
3. extract the information of all the port name, port direction, port width of all the entity
4. write the content in to VHDL_port.txt

example:4

1. Read the VHDL.txt file
2. Read line by line
3. extract the information of all the port name, port direction, port width of all the entity
4. write the content in to VHDL_port.txt

```
set file [open "VHDL.txt" r]
```

```
set file1 [open VHDL_port_details.txt" w]
```

```
while {[gets $file line] >= 0} {
```

```
if {[regexp -nocase { *([a-z0-9_ ]*) *: *(in|out) +std_logic_Vector(((0-9)+) +downto +((0-9)+))} $line match pn pd upw lpw]} {  
puts "port name is :$pn port direction is:$pd port width is:[expr ($upw - $lpw) + 1]"  
puts $file1 "port name is :$pn port direction is:$pd port width is:[expr ($upw - $lpw) + 1]"  
} elseif {[regexp -nocase { *([a-z0-9_ ]*) *: *(in|out) +std_logic} $line match pn pd]} {  
puts "port name is :$pn port direction is:$pd port width is:1"  
puts $file1 "port name is :$pn port direction is:$pd port width is:1"  
}  
}
```

```
set a "alu_result: out std_logic_vector(15 downto 0)"
```

```
set b [regexp -nocase { *([a-z_ ]*) *: *(in|out) +std_logic_Vector(((0-9)+) +downto +((0-9)+))} $a match portname portdirection upw lpw]
```

```
puts "port name is:$portname"
```

```
puts "port direction is:$portdirection"
```

```
puts "port width is :[expr ($upw - $lpw) + 1]"
```

```
set a "clk : in std_logic"
```

```
set b [regexp -nocase { *([a-z_ ]*) *: *(in|out) +std_logic} $a match portname  
portdirection]
```

Tcl tool command language

```
puts "port name is:$portname"
puts "port direction is :$portdirection"
puts "port width is :1"
```

```
set a "mem_write_en,mem_read:in std_logic"
```

```
set b [regexp -nocase { *([a-z_, ]*) *: *(in|out) +std_logic} $a match portname
portdirection]
```

```
puts "port name is:$portname"
puts "port direction is:$portdirection"
puts "port width is:1"
```

example:5

for practise text file extract all the file names which end with .ctl files.

```
set file [open "practise.txt" r]
set file1 [open "practise_file_name.txt" w]

while {[gets $file line] >= 0} {

if {[regexp -nocase {(+.ctl)} $line match fn]} {
puts $fn
puts $file1 $fn
}
}
```

example:6

1. access all the 5 files(code1_data_1,2,3,4,5)
- 2.pass the argument as nom_temperature
- 3.to match this write the regular exprssion
4. create a new file that conatins filename with value of the nom_tempearture
5. if we pass wrong argument print the error.
6. if nom_temperature value is not aviable print output as : --

```
lassign $argv name
```

```
set files [glob C/Users/kumar/Desktop/TCL_March_2023 "code1_data_*.txt"]
set file1 [open "nom_temp_details.txt" w]
```

```
foreach f $files {
```

Tcl tool command language

```
set fh_read [open $f r]

while {[gets $fh_read line] >= 0} {

set b [regexp -nocase { *([a-z_]*) *: *([a-z0-9.-]+) *};] $line match nor_tem
nor_temp_value]
if {$b == 1 && $name == $nor_tem} {
puts "file name is $f and its value is $nor_temp_value"
puts $file1 "file name is $f and its value is $nor_temp_value"
}
}
}

lassign $argv name

set count 0
set count1 0

set files [glob C/Users/Lenovo/Desktop/TCL_Sep_Oct-2022 "code1_data_*.txt"]
set fh_write [open "file_new.txt" w]
foreach f $files {
set fh_read [open $f r]
set flag 0
set flag1 0
while {[gets $fh_read line]>=0} {
    set b [regexp -nocase { *([a-z_]*) *: *([a-z0-9.-]+) *};] $line temp myname val]
    set c [regexp -nocase { *([a-z][a-z_]*) *: *("[a-z0-9.-]+") *};] $line temp1 myname1 val1]
    if {$b ==1 && $name == $myname} {
        puts "$f : $val"
        puts $fh_write "$f : $val"
        incr flag
        incr count
    } elseif {$c == 1 && $name == $myname1} {
        puts "$f : $val1"
        puts $fh_write "$f : $val1"
        incr flag1
        incr count1
    }
}

}
if {$flag == 0 && $flag1 == 0} {
    puts "$f : --"
    puts $fh_write "$f : --"
}
}
```


Tcl tool command language

```
if {$count == 0 && $count1 == 0} {  
    puts "Error: Wrong argument"  
}
```